Query operators

- SELECTION
  - Sequential scan [the whole relation is scanned]
  - Index scan [one or multiple indices are scanned]
    - [optional] followed by fetching of matching tuples
    - [optional] followed by additional selection conditions
- PROJECTION
  - Usually performed in memory

EXTERNAL SORTING

- A large number of operators can be executed by an intermediate sorting step:
  - DISTINCT
  - ORDER BY
  - GROUP BY
  - UNION/INTERSECTION/DIFFERENCE

EXTERNAL SORTING

- A limited amount of memory is available to the sort operation
  - K: the number of memory pages available for the sort operation
  - NCARD(R): total number of disk pages for relation R
- The sort operation is a two step process
  - STEP 1: Sort chunks of M blocks in memory and write each block to disk
  - STEP 2: Merge the chunks in successive steps into a single sorted relation

EXTERNAL SORTING – STEP 1

```
do {
    read M pages from disk into main memory
    sort them in memory with one of the known methods (assume that,
          apart from the M-page buffer, additional memory is available
          for in-memory sorting)
    dump the sorted file segment into a new file}
} until (end-of-file)
```

I/O COST: The relation is read once and written once, so the cost is:

2* NCARD(R)

EXTERNAL SORTING – STEP 2

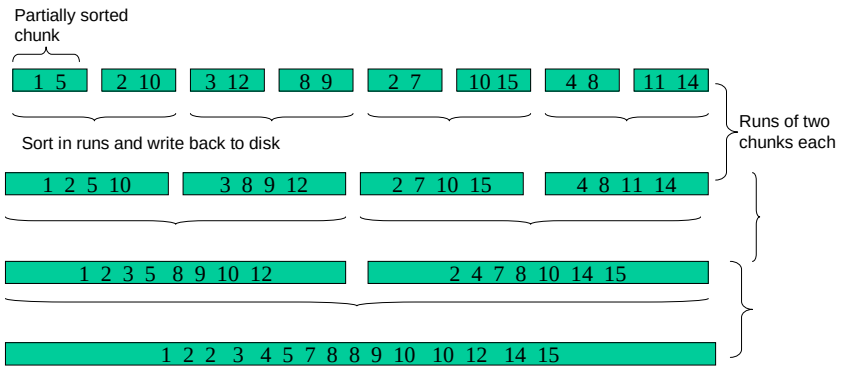
while (there are non empty input runs) {
    choose a smallest tuple (with respect to the sort key) in each run,
        and output the smallest among these
    delete the chosen tuple from the respective input run
}

The upper bound of this step depends on how many runs it will take sort the whole relation.

There are R= ceiling(NCARD(R)/M) groups, at each step the whole relation is read once and written once. There can at most be R runs, so the upper bound on the total cost is: 2*R*NCARD(R)

If R=K, then the second step will take only one more read over the relation, i.e. NCARD(R), hence the total cost of sorting will be 3*NCARD(R) .

EXTERNAL SORTING – STEP 2



Total of 3 runs: The relation is read once and written once in the first two runs, and finally read once in the last run. Total processing cost is: 5*NCARD(R)

EXTERNAL SORTING – STEP 2

- In the second step, suppose we are merging K chunks into one.
- We read from each chunk the first block into memory, and perform the merge.
- During merge, output tuples are deleted from the corresponding blocks (see the algorithm for step 2).
- Whenever the in-memory block of a chunk becomes empty, we read the next tuple (in the sorted order) of this chunk.
- When all chunks are empty, the process is complete.

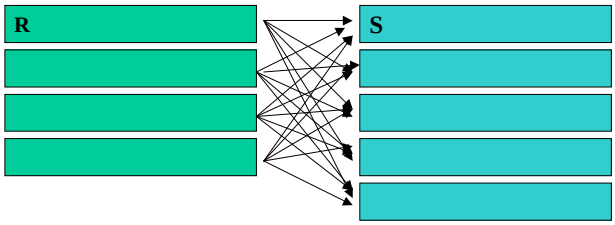
Joins

- Joins usually require multiple passes over the relations
- We will consider equijoins here
- Suppose relations T and R have the attribute A in common
- R ▷◁ S finds all tuples r and s from R and S where r.A = s.A, and computes a result tuple r+s.
- Nested loop join
  - select one relation as “outer” and the other as “inner”
  - do only one pass over the outer relation
  - for each tuple o in the outer relation
    find all tuples i from the inner relation where o.A = i.A
    output o+i

for each fixed value o.A, compute
SELECT * FROM i WHERE o.A = i.A

Nested loop join

- Join relation R with relation S
- Relation R has 4 tuples and relation S has 5 tuples
- Relation R is the outer relation



Done

Nested loop join (R ▷◁ S)

- The cost of the nested loop join depends on
  - the size of the outer relation (it is read only once)
  - number of distinct values for the column A in the outer relation
  - the access methods available on the inner relation
  - number of tuples to be retrieved from the inner relation for each distinct value of column A
- If the inner relation (S) can fit in memory, then the total cost of nested loop join is NCARD(R) + NCARD(S)
- If the inner relation (S) cannot fit in memory, then it has to read completely in memory for each block of R. Then, the total cost is NCARD(R) + (NCARD(R) * NCARD(S))

Estimating the size of join

- Estimating the size of R ▷◁ S on R.A = S.A
  - Let T = RxS, the estimated size of T is
    CARD(R) * CARD(S)
  - Estimate SELECT R.A = S.A FROM T:
    - For any given tuple r of R, how many tuples s in S where r.A = s.A?
      - Note that, for any given tuple r, r.A is a constant, say "c"
      - The filter factor for s.A=c is given by 1/COLCARD(S.A)
    - For any given tuple s of S, how many tuples r in R where r.A = s.A?
      - The filter factor for r.A=c is given by 1/COLCARD(R.A)
    - The filter factor can be chosen to be the minimum of these: 1/MAX(COLCARD(R.A),COLCARD(S.A))
  - The expected size of R ▷◁ S on R.A = S.A is then given by :

CARD(R) * CARD(S) * 1/MAX(COLCARD(R.A),COLCARD(S.A))

Estimating the size of join

Project(EmpNo,ProjNo, Name, Description, StartDate, SprNo)
Employee(EmpNo, Name, Office, Phone, Salary)
Foreign key: Project.SprNo → Employee.EmpNo

Card(Project) = 10,000 Ncard(Project) = 500
ColCard(Project.SprNo) = 500 ColCard(Project.EmpNo)= 8,000
ColCard(Project.ProjNo) = 100
Card(Employee) = 10,000,000 Ncard(Employee) = 10,000

Project ▷◁ Employee on SprNo = EmpNo
Project ▷◁ Employee on EmpNo = EmpNo
Project ▷◁ Project on EmpNo = SprNo
Project ▷◁ Project on EmpNo = EmpNo AND ProjNo <> ProjNo

Estimating the size of join

T = Project ▷◁ Employee on SprNo = EmpNo
- Every supervisor can join with atmost one Employee
- Card(T) = Card(Project)

T = Project ▷◁ Employee on EmpNo = EmpNo
- Every employee in Project can join with atmost one Employee
- Card(T) = Card(Project)

T = Project ▷◁ Project on EmpNo = SprNo
- Given an employee p.EmpNo, how many tuples can there be with that employee as supervisor? FF= 1/500. Card(T) = 10,000 * 10,000 * 1/500
- Given a supervisor p.SprNo, how many tuples can there be with that supervisor working on projects? FF = 1/8,000. Card(T) = 10,000 * 10,000 * 1/8,000 = 12,500
- Choose the minimum 12,500

T = Project ▷◁ Project on EmpNo = EmpNo AND ProjNo <> ProjNo
- What is the filter factor of this selection condition? FF = 1/8,000 * 99/100
- The expected cardinality is then 10,000*10,000*FF = 12,375

Nested loop join (R ▷◁ S)

- Using indices with nested loop join:
  - For each tuple of outer relation (R) in memory, retrieve the matching tuples from S using an index
  - This method can be useful if the expected number of tuples in the output is much than the number of tuples in S
- Example:
  - Project(EmpNo,ProjNo, Name, Description, StartDate, SprNo)
  - Employee(EmpNo, Name, Office, Phone, Salary)

Foreign Key: Project.SprNo → Employee.EmpNo

Card(Project) = 10,000 Ncard(Project) = 500
ColCard(Project.SprNo) = 500 ColCard(Project.EmpNo)= 8,000
Card(Employee) = 10,000,000 Ncard(Employee) = 10,000

Project ▷◁ Employee on SprNo = EmpNo
Project ▷◁ Employee on EmpNo = EmpNo

Hash Join

- Given R join S on R.A=S.A
  - STEP 1
    - Read all tuples of R and S into memory and hash into M buckets using A as hash key.
    - Whenever a bucket (for some hash value j) is full, write that bucket to disk.
    - COST: The relations are read once and written once, hence 2*NCARD(R)+2*NCARD(S)
  - STEP 2
    - Read the buckets into memory one by one, and perform the join within the tuples in the same bucket
    - COST: Depends on whether each bucket can be read completely to the memory, then the cost is NCARD(R)+NCARD(S)

Merge sort join

- Suppose relations T and R have the attribute A in common
- T $\triangleright\triangleleft$ R finds all tuples t and r from T and R where t.A = r.A, and computes a result tuple t+r.
- Merge sort join consists of two steps
  - sort of the two relations on the join attributes
  - join of the tuples from the two tables (this step normally requires only one pass over both the outer and inner relations)

Merge Sort Join Algorithm

```
MERGESORTJOIN(T and R on column A)
  Sort T and R on disk with respect to attribute A
  Load the first page of T and R into memory
  Set pointers Tp, Rp to the first tuples of T and R in memory
  while end of both relations is not reached
      If Tp.A < Rp.A then advance Tp to the next tuple of T
      If Tp.A > Rp.A then advance Rp to the next tuple of R
      If Tp.A = Rp.A then output the join of Tp and Rp
          set Temp = Rp
          while (Tp.A = Rp.A)
              {output join of Tp and Rp, advance Rp }
          advance Tp to the next tuple, reset Rp to Temp
```

Advancing a pointer may mean reading a new disk page

Merge sort join revisited

- In fact, the merge sort join can be combined with the external sort algorithm
- Apply STEP 1 of merge sort to each relation
- Read partially sorted relations in runs and while merging, apply the join criteria.

Cost of Merge Sort Join

- Cost = T1 + T2, where
  - T1 is time to sort both relations
  - T2 is time to do the join
- If no duplicate values in join, one pass over both relations
- Suppose disk page can store up to Z tuples of T or R
  - Assume we are using only two memory pages to hold one page for T and one page for R
  - We make a total of FF(R.A=T.A)*card(R) / Z disk reads for each different join value (read one tuple of T, read all joining values of R, then read the next tuple of T)
  - Total disk reads are card(T)/Z * FF(R.A=T.A)*card(R) / Z
- The total number of different values in T is 1/FF(T.A).
- So, T2 = card(T)/Z + card(T)*card(R)*FF(R.A)/Z^2 disk reads

Query Processing

- Parse a query
  - Verify its syntax
  - Verify the existence of relations, attributes (data dictionary)
  - Convert into a relational algebra expression

Given

SELECT TargetList
FROM Rel1, …, Reln
WHERE Condition

Derive:

((Rel1 x … x Reln) WHERE Condition)[TargetList]

Simplify conditions in the queries as much as possible to remove unsatisfiable, unnecessary conditions.

Heuristic optimizations

Use algebraic equivalences:

- R WHERE cond1 AND cond2 ≡ (R WHERE cond1) WHERE cond2
- (R WHERE cond1) WHERE cond2 ≡ (R WHERE cond2) WHERE cond1
- R[attr] ≡ (R[attr']) [attr] if attr ⊆ attr'
- (R WHERE cond) [attr] ≡ (R[attr]) WHERE cond if attr contains all attributes needed to evaluate cond
- Cartesian product and join are both commutative and associative.

General rules of thumb

- Joining smaller relations is better than joining large ones
- Performing equi-join is better than a Cartesian product

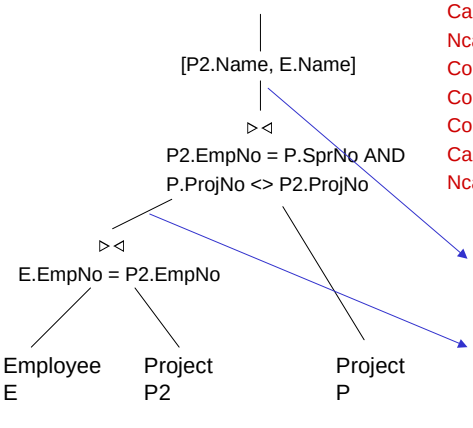
Heuristic optimizations

- Push selections, projections through joins and Cartesian products as much as possible.
  - (R x S) WHERE Cond ≡ R $\triangleright\triangleleft_{Cond}$ S if Cond involves both R and S
  - (R x S) WHERE Cond ≡ (R WHERE Cond) x S if Cond involves only attributes in R
  - (R $\triangleright\triangleleft$ S) WHERE Cond ≡ (R WHERE Cond) $\triangleright\triangleleft$ S same as above
  - (R $\triangleright\triangleleft$ S) [attr] ≡ (R[attr'] $\triangleright\triangleleft$ S) [attr] if attr' are attributes of R that contain all join attributes and attr' ∪ head(S) ⊇ attr.
  - Similar condition for Cartesian product exists.

Query optimization

- Parse query into an algebra expression (called query tree)
- Find the space of
  - all possible heuristic optimizations available
  - all possible implementation of query operators available (which indices exist, are relations already sorted?)
  - all possible ordering of joins possible
  - which operators can be combined into a single operation (pipelined)
- A query plan contains: ordering of all operations, ordering assignment of joins, assignment of implementations to operators and optional operators for materialization of intermediate results
- Most query optimizers will perform a cost-based search within the space of all possible query plans using the size estimates of intermediate results.

SELECT P2.Name PrName, E.Name EmployeeName
FROM project P, employee E, project P2
WHERE P.SprNo = P2.EmpNo AND P.ProjNo <> P2.ProjNo AND
P2.EmpNo = E.EmpNo

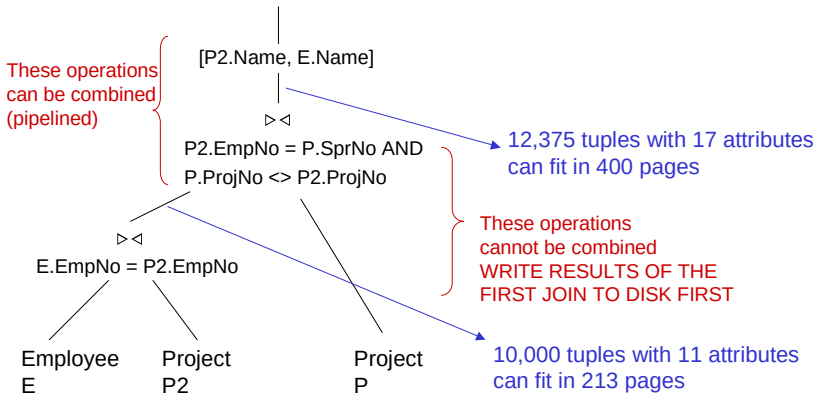


Card(Project) = 10,000
Ncard(Project) = 500
ColCard(Project.SprNo) = 500
ColCard(Project.EmpNo)= 8,000
ColCard(Project.ProjNo) = 100
Card(Employee) = 10,000,000
Ncard(Employee) = 10,000

Expected # of tuples: 12,375
How many pages?

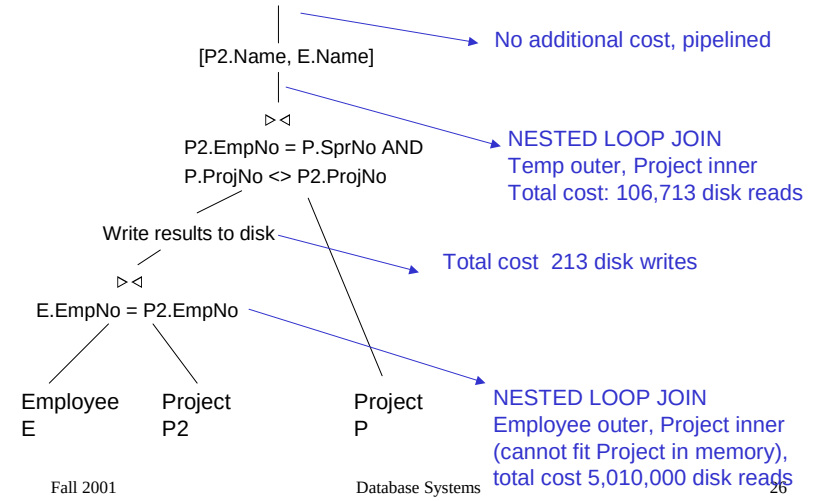
Expected # of tuples: 10,000
How many pages do we need to store the result?

Suppose we have 200 memory pages available for this query
Assume also that a page can store 512/k tuples of a relation with k attributes (obviously the relations below are stored more sparsely than this, but we will use this assumption for the intermediate results)



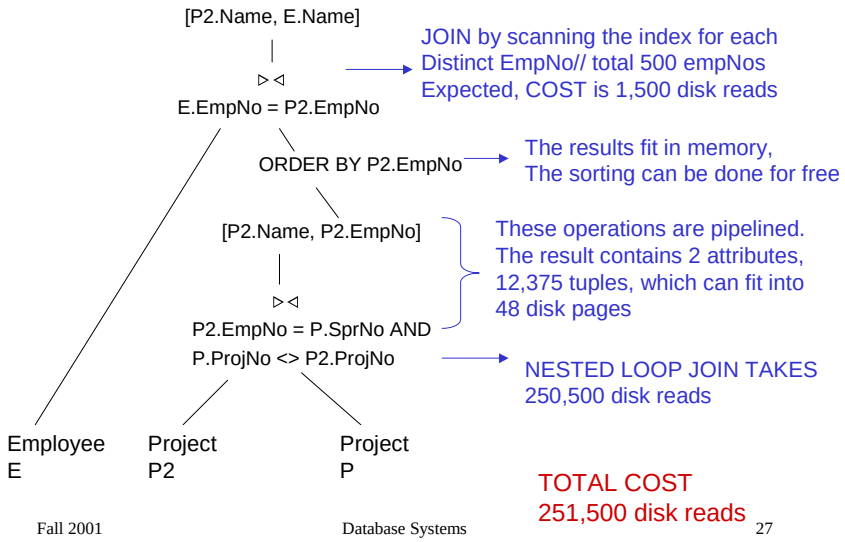
Suppose we have 200 memory pages available for this query

TOTAL COST
5,116,926 disk reads



New Plan

Assume an index on E.EmpNo, assume also that each equality search takes exactly 3 disk reads



Cost based query optimization

- Enumerate all possible join orderings and query plans
  - Find the cost of each plan
  - Eliminate plans that are guaranteed to be loosers
  - Continue checking other plans
- The statistics can sometimes be misleading during query optimization
  - Suppose there are two values for an attribute A of a relation R with 10,000 tuples, but only two tuples for R.A=1 and 9,998 for R.A=2.
  - It is possible to give hints to the optimizer during query processing

```
SELECT /*+ FULL(P) */
            P2.Name PrName, E.Name EmployeeName
FROM project P, employee E,  project P2
WHERE  P.SprNo = P2.EmpNo AND P.ProjNo <> P2.ProjNo AND
            P2.EmpNo = E.EmpNo
```